

Trabajo Práctico N° 3

Resolver problemas en Multiparadigmas

1. Buscar elemento en una colección
2. Ordenar elementos en una colección
3. Gestionar una lista de tareas pendientes (to-do list), permitiendo:
 - Agregar nuevas tareas con título y prioridad
 - Marcar tareas como completadas
 - Mostrar las tareas pendientes ordenadas por prioridad

❖ Buscar elemento en una colección y ordenar elementos en una colección

```
# PARADIGMA IMPERATIVO
numeros = [30, 10, 50, 20, 40]
def buscar_elemento(coleccion, elemento):
    for i in range(len(coleccion)):
        if coleccion[i] == elemento:
            return i
    return -1
posicion = buscar_elemento(numeros, 20)

if posicion != -1:
    print("El elemento fue encontrado en la posición:",
posicion)
    print("El elemento no fue encontrado")

# 2. Ordenar elementos en una colección
def ordenar_elementos(coleccion):
    return sorted(coleccion)

numeros_ordenados = ordenar_elementos(numeros)
print("Números ordenados:", numeros_ordenados)
```

```

# PARADIGMA ORIENTADO A OBJETOS
class ColeccionNumeros:
    def __init__(self, numeros):
        self.numeros = numeros

    def buscar(self, elemento):
        if elemento in self.numeros:
            return self.numeros.index(elemento)
        return -1

# 2. Ordenar elementos en una colección
def ordenar(self):
    return sorted(self.numeros)

coleccion = ColeccionNumeros([30, 10, 50, 20, 40])
posicion = coleccion.buscar(20)
if posicion != -1:
    print("El elemento fue encontrado en la posición:",
          posicion)
    print("El elemento no fue encontrado")

print("Números ordenados:", coleccion.ordenar())

```

```

# PARADIGMA FUNCIONAL
def buscar_elemento(coleccion, elemento):
    posiciones = list(filter(lambda i: coleccion[i] == elemento, range(len(coleccion))))

    if len(posiciones) > 0:
        return posiciones[0]
    return -1

# 2. Ordenar elementos en una colección
def ordenar_elementos(coleccion):
    return sorted(coleccion)

posicion = buscar_elemento(numeros, 20)

if posicion != -1:
    print("El elemento fue encontrado en la posición:", posicion)
else:
    print("El elemento no fue encontrado")

print("Números ordenados:", ordenar_elementos(numeros))

```

```

# PARADIGMA LOGICO
% Colección de números
numero(30).
numero(10).
numero(50).
numero(20).
numero(40).

buscar(X) :- elemento(X)

?- buscar(20).
% Respuesta esperada:
% true.

# 2. Ordenar elementos en una colección
% ?- findall(X, numero(X), Lista), sort(Lista, Ordenada).
% Ordenada = [10, 20, 30, 40, 50].

```

❖ Gestionar lista de tareas pendientes

```

# PARADIGMA IMPERATIVO
tareas = []
def agregar_tarea(titulo, prioridad):
    tarea = {
        "titulo": titulo,
        "prioridad": prioridad,
        "completada": False
    }
    tareas.append(tarea)

def marcar_completada(titulo):
    for tarea in tareas:
        if tarea["titulo"] == titulo:
            tarea["completada"] = True

def mostrar_pendientes():
    pendientes = []

    for tarea in tareas:
        if tarea["completada"] == False:
            pendientes.append(tarea)

    pendientes_ordenadas = sorted(pendientes, key=lambda tarea: tarea["prioridad"])

    for tarea in pendientes_ordenadas:
        print(tarea["titulo"], "- Prioridad:", tarea["prioridad"])

agregar_tarea("Estudiar lenguajes", 2)
agregar_tarea("Hacer el trabajo práctico", 1)
agregar_tarea("Leer apuntes", 3)

marcar_completada("Leer apuntes")

print("Tareas pendientes:")
mostrar_pendientes()

```

```

# PARADIGMA ORIENTADO A OBJETOS
class Tarea:
    def __init__(self, titulo, prioridad):
        self.titulo = titulo
        self.prioridad = prioridad
        self.completada = False

    def marcar_completada(self):
        self.completada = True

class GestorTareas:
    def __init__(self):
        self.tareas = []

    def agregar_tarea(self, titulo, prioridad):
        nueva_tarea = Tarea(titulo, prioridad)
        self.tareas.append(nueva_tarea)

    def marcar_completada(self, titulo):
        for tarea in self.tareas:
            if tarea.titulo == titulo:
                tarea.marcar_completada()

    def mostrar_pendientes(self):
        pendientes = []

        for tarea in self.tareas:
            if tarea.completada == False:
                pendientes.append(tarea)

        pendientes_ordenadas = sorted(pendientes, key=lambda tarea: tarea.prioridad)

        for tarea in pendientes_ordenadas:
            print(tarea.titulo, "- Prioridad:", tarea.prioridad)

gestor = GestorTareas()

gestor.agregar_tarea("Estudiar lenguajes", 2)
gestor.agregar_tarea("Hacer el trabajo práctico", 1)
gestor.agregar_tarea("Leer apuntes", 3)

gestor.marcar_completada("Leer apuntes")

print("Tareas pendientes:")
gestor.mostrar_pendientes()

```

```

# PARADIGMA FUNCIONAL
def crear_tarea(titulo, prioridad):
    return {
        "titulo": titulo,
        "prioridad": prioridad,
        "completada": False
    }

def agregar_tarea(tareas, titulo, prioridad):
    nueva_tarea = crear_tarea(titulo, prioridad)
    return tareas + [nueva_tarea]

def marcar_completada(tareas, titulo):
    return list(
        map(
            lambda tarea: {
                "titulo": tarea["titulo"],
                "prioridad": tarea["prioridad"],
                "completada": True
            } if tarea["titulo"] == titulo else tarea,
            tareas
        )
    )

def obtener_pendientes_ordenadas(tareas):
    pendientes = filter(lambda tarea: tarea["completada"] == False, tareas)
    return sorted(pendientes, key=lambda tarea: tarea["prioridad"])

tareas = []

tareas = agregar_tarea(tareas, "Estudiar lenguajes", 2)
tareas = agregar_tarea(tareas, "Hacer el trabajo práctico", 1)
tareas = agregar_tarea(tareas, "Leer apuntes", 3)

tareas = marcar_completada(tareas, "Leer apuntes")

print("Tareas pendientes:")

for tarea in obtener_pendientes_ordenadas(tareas):
    print(tarea["titulo"], "- Prioridad:", tarea["prioridad"])

```



```
#PARADIGMA LÓGICO
% Colección de números
numero(30).
numero(10).
numero(50).
numero(20).
numero(40).

% 1. Buscar elemento en una colección
?- numero(20).
true.

% 2. Ordenar elementos en una colección
?- findall(X, numero(X), Lista), sort(Lista, Ordenada).
% Lista = [30, 10, 50, 20, 40],
% Ordenada = [10, 20, 30, 40, 50].

% 3. Gestionar una lista de tareas pendientes
tarea(Titulo, Prioridad, Estado)
tarea("Estudiar lenguajes", 2, pendiente).
tarea("Hacer el trabajo practico", 1, pendiente).
tarea("Leer apuntes", 3, completada).

% Regla para saber si una tarea está pendiente
tarea_pendiente(Titulo, Prioridad) :-
    tarea(Titulo, Prioridad, pendiente).

% Consulta:
% ?- tarea_pendiente(Titulo, Prioridad).
% Titulo = "Estudiar lenguajes",
% Prioridad = 2 ;
% Titulo = "Hacer el trabajo practico",
% Prioridad = 1.

% Para mostrar tareas pendientes ordenadas por prioridad:
% ?- findall(Prioridad-Titulo, tarea_pendiente(Titulo, Prioridad),
% Lista), sort(Lista, Ordenada).
% Ordenada = [1-"Hacer el trabajo practico", 2-"Estudiar lenguajes"].
```

Criterios de comparación: armar tabla comparativa con los siguientes criterios y asignar valores cualitativos (ej. ALTA, MEDIA, BAJA) o cuantitativos (ej. 1-5) para visualizar rápidamente las fortalezas y debilidades de cada paradigma frente al mismo problema

- Claridad y legibilidad del código
 - ¿El código es fácil de leer y entender?
 - ¿La estructura refleja bien el paradigma?
- Nivel de abstracción
 - Procedural: funciones y estructuras básicas.
 - Orientado a objetos: clases, encapsulación, herencia.
 - Funcional: inmutabilidad, funciones puras, composición.
 - Lógico: hechos y reglas, consultas declarativas.
- Eficiencia y rendimiento
 - ¿Cómo maneja memoria y tiempo de ejecución?
 - ¿El paradigma facilita optimizaciones?
- Facilidad de mantenimiento y escalabilidad
 - ¿Es sencillo agregar nuevas funcionalidades?
 - ¿El paradigma favorece modularidad o reutilización?
- Expresividad y concisión
 - ¿Cuánto código se necesita para expresar la solución?
 - ¿El paradigma permite expresar la lógica de manera directa?

Completar tabla: cada paradigma se ajusta mejor a un tipo de problema

Paradigma	Problema 1	Problema 2	Problema 3	Problema 4	Problema 5
Procedural	cálculos y algoritmos paso a paso				
OO	sistemas con entidades y relaciones				
Funcional	procesamiento de datos y transformaciones				
Lógico	problemas declarativos y de inferencia				
Multi-paradigma	app modernas que requieren flexibilidad				

❖ PROBLEMA 1

CRITERIOS	Imperativo	Orientado a objetos	Funcional	Lógico
Claridad y legibilidad del código	ALTA	ALTA	MEDIA	MEDIA
Nivel de abstracción	MEDIO	ALTO	ALTO	ALTO
Eficiencia y rendimiento	ALTA	MEDIA	MEDIA	BAJO
Facilidad de mantenimiento y escalabilidad	MEDIA	ALTA	ALTA	MEDIA
Expresividad y concisión	MEDIA	MEDIA	ALTA	ALTA

❖ **PROBLEMA 2**

CRITERIOS	Imperativo	Orientado a objetos	Funcional	Lógico
Claridad y legibilidad del código	BAJA	BAJA	MEDIA	MEDIA
Nivel de abstracción	BAJO	ALTO	ALTO	ALTO
Eficiencia y rendimiento	MEDIA	MEDIA	ALTA	BAJA
Facilidad de mantenimiento y escalabilidad	MEDIA	ALTA	ALTA	MEDIA
Expresividad y concisión	BAJA	MEDIA	ALTA	ALTA

❖ **PROBLEMA 3**

CRITERIOS	Imperativo	Orientado a objetos	Funcional	Lógico
Claridad y legibilidad del código	ALTA	ALTA	BAJA	MEDIA
Nivel de abstracción	MEDIO	ALTO	ALTO	ALTO
Eficiencia y rendimiento	ALTA	BAJA	BAJA	BAJA
Facilidad de mantenimiento y escalabilidad	BAJA	ALTA	ALTA	MEDIA
Expresividad y concisión	BAJA	MEDIA	ALTA	ALTA